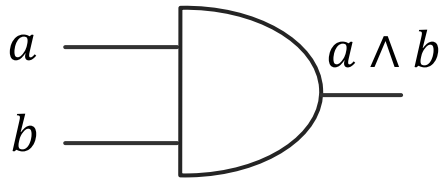
Proiectare Logică

I CB: Mariana Mocanu mariana.mocanu@upb.ro
I CC: Anca Morar anca.morar@upb.ro
I CA & CD: Costin Chiru costin.chiru@upb.ro

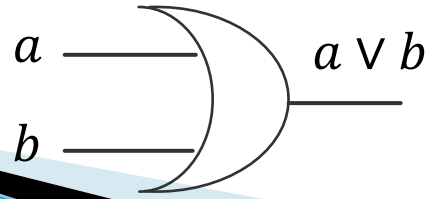
Curs 2: Algebra booleană. Constante booleene, variabile booleene și funcții booleene. Seturi complete de operatori.

Fundamentele matematice ale proiectării logice – Algebra Booleană

- O algebră booleeană se defineşte ca un triplet (B,^,V) unde B este o mulțime de valori binare, iar ^ (și), V (sau) sunt operatori binari pe B a.î. să fie adevărate următoarele proprietăți:
  - P1: Pe B, ^ şi V sunt comutative;
    - ∀ a, b ∈ B: a ^ b = b ^ a și a V b = b V a



AND Circuit „ȘI” (^, •)



OR Circuit „SAU” (V, +)

Fundamentele matematice ale proiectării logice – Algebra Booleană

- P2: În B există elemente distincte de identitate 1 şi 0 relative la ^ respectiv V;
  - ∀ a ∈ B: a ^ 1 = a și a V 0 = a
- P3: ^ şi V sunt distributive unul față de altul;
  - ∀ a, b, c ∈ B: a ^ (b V c) = (a ^ b) V (a ^ c) și
  - ∀ a, b, c ∈ B: a V (b ^ c) = (a V b) ^ (a V c)
- P4: Pentru oricare a ∈ B există ā a.i.
  - a ^ ā = 0 și
  - a V ā = 1;
- **P1–P4 sunt considerate AXIOMELE algebrei booleene**

Algebra Booleană

- Algebra booleană nu e decât o particularizare peste B relativ la proprietățile definite
- Unele concepte sunt cunoscute de la programare:
  - Tipuri de date: numerice, logice, text
  - Operatori: numerici, logici, relaționali
- Diferențe/asemănări între operatori logici și relaționali: cei relaționali operează pe date numerice și întorc rezultate logice

Teoremele algebrei booleene (1)

- T1: Elementul ā asociat cu un element $a \in B$ prin proprietatea P4 este unic.
  - Dem 1: prin reducere la absurd (demonstrația matematică)
    - Pp. $\exists x, y \in B, x \neq y$ a. î.
      - $a \wedge x = 0,\ a \vee x = 1$
      - $a \wedge y = 0,\ a \vee y = 1$
    - Utilizând P2 → $x = y$
  - Dem 2: găsire de proprietăți matematice ce pot fi exploatate
  - Dem 3: metoda verificării comportamentului → nu e metodă matematică, dar e ușor de adaptat la PL (prin tabela de adevăr)

În curs sunt foarte puține lucruri care trebuie memorate, restul se pot deduce una din alta! → Nu e important numărul teoremei, ci să știți să o aplicați!

Teoremele algebrei booleene (2)

- T2: Fie a ∈ B; atunci a V a = a și a ^ a = a
- T3: Fie a ∈ B; atunci a V 1 = 1 și a ^ 0 = 0
- T4: Legea de absorbție:
  - a V (a ^ b) = a
  - a ^ (a V b) = a
- T5: V şi ^ sunt asociative:
  - a ^ (b ^ c) = (a ^ b) ^ c
  - a V (b V c) = (a V b) V c
- T6: Principiul dublei negări: ∀ a ∈ B, $\bar{\bar{a}} = a$

Teoremele algebrei booleene (3)

- T7: $\bar{0} = 1 \;\; \text{și} \;\; \bar{1} = 0$
- T8: Regulile lui de Morgan:

$$\overline{a \wedge b} = \bar{a} \vee \bar{b}$$

$$\overline{a \vee b} = \bar{a} \wedge \bar{b}$$

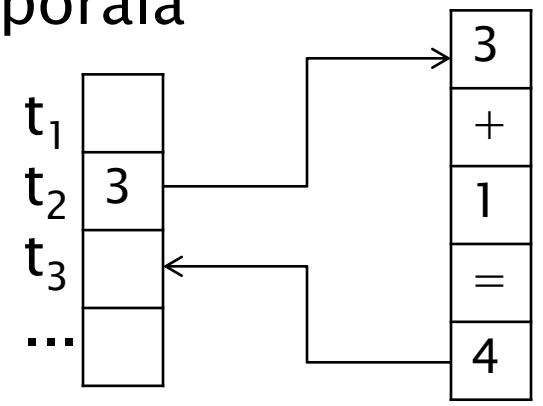
- Generalizare de Morgan: Fie $x_1, x_2, \ldots x_n \in B$:

$$\overline{x_1 \wedge x_2 \wedge \cdots \wedge x_n} = \overline{x_1} \vee \overline{x_2} \vee \cdots \vee \overline{x_n}$$

$$\overline{x_1 \vee x_2 \vee \cdots \vee x_n} = \overline{x_1} \wedge \overline{x_2} \wedge \cdots \wedge \overline{x_n}$$

Convenție

- $\bar{}$ = !, V = + iar ^ = ·
- Observație: sunt operatori logici, nu matematici!
  - Ex: 1 + 1 = 1
- Unde ați mai întâlnit diferențe față de matematică?
  - Incrementare: x = x + 1
  - De ce e corect? Are conotație temporală



La ce sunt utile teoremele?

- 2 aspecte:
  - Matematic: expresii, funcții, reducerea (**minimizarea**) acestora
  - Practic: asociem fiecărei funcții un anumit circuit → punând laolaltă diverse dispozitive în diferite feluri putem genera diverse funcționalități, dar e important câte astfel de dispozitive se folosesc, mai ales având în vedere T4 și T6

$$1 = \ln\left(\lim_{z\to\infty}\left(\left(\left(\overline{X}^{T}\right)^{-1} - \left(\overline{X}^{-1}\right)^{T} \right)! + \frac{1}{z} \right)^{2} \right)$$

Constante booleene, variabile booleene, funcții booleene

- Într-o algebră booleană, 0 și 1 sunt singurele constante.
- O variabilă logică este o mărime logică ce ia o valoare ce se poate schimba.
- O expresie logică este o succesiune de variabile/constante unite prin operatori, respectând regulile de evaluare ale operatorilor
  - a,b ∈ B, ab+ nu e expresie, a+b e expresie
- O funcție booleană de n variabile x_1, x_2, ... x_n, notată $f(x_1, x_2, \ldots x_n)$ se definește ca o aplicație a lui B^n (B x B x B x… B) → B. Variabilele x_1, x_2, ... x_n sunt independente
- O funcție booleană = rezultatul oricărei combinații finite de constante și variabile realizate cu ajutorul operatorilor !, ·, +

Forme de reprezentare a funcțiilor booleene

- 1. Algebrică: se definește funcția pe baza unei expresii:
  - Teoremele învățate ne ajută la reprezentarea algebrică $f(x_1, x_2, \ldots x_n) =$ expresie
  - Ex: $f(a, b) = a + b$
  - Expresiile pot fi combinate → forme relativ complicate
- 2. Tabel de adevăr
  - (-) Foarte multe combinații dacă funcția depinde de multe variabile
  - (+) Se cunoaște rezultatul pentru fiecare combinație în parte
- 3. Grafică – diagrame Karnaugh
- 4. Zecimală – asociază fiecărui monom o valoare zecimală și permite automatizarea minimizării
- Implementarea – dispozitive (elemente fizice folosite în implementarea circuitului): AND, OR, NOT, NAND...

Reguli de precedență a operatorilor

- Evaluarea unei expresii se face respectând anumite reguli (reguli de precedență a operatorilor):
  - 1. se evaluează, mai întâi, toate subexpresiile conținute în paranteze, evaluarea pornind de la zona cea mai imbricată (folosind regulile de mai jos) : a • (!b + (a • c))
  - 2. se aplică toți operatorii NEGAȚIE (NOT, !, ¯): a + b + c • !b + !c
  - 3. Se aplică toți operatorii ȘI (^, •): a + b • c
  - 4. Se aplică toți operatorii SAU (V , +)
  - 5. Evaluarea se face de la stânga la dreapta dacă avem același operator peste tot

Reprezentarea funcțiilor booleene prin tabele de adevăr

- Redă într-o formă ordonată funcționalitatea unei funcții pentru fiecare combinație de intrare posibilă (2^N combinații)

x_1	x_2	…	x_n	f
0	0	…	0	f (0, 0, … 0)
0	0	…	1	f (0, 0, … 1)
1	1	…	1	f (1, 1, … 1)

- Pe fiecare linie există o singură valoare logică

- De exemplu, $f_1(a) = \bar{a}$

a	$f_1(a) = \bar{a}$
0	1
1	0

Reprezentarea funcțiilor booleene prin tabele de adevăr

- Alte exemple:
  - $f_2(a, b) = a \wedge b = a \bullet b$ (AND)

a	b	$a \wedge b$	$a \vee b$
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	1

  - $f_3(a, b) = a \vee b = a + b$ (OR)
- De unde știm că aceste valori sunt așa și nu altfel?
  - Din P1-4 și din teoreme

Reprezentarea funcțiilor booleene prin tabele de adevăr

- Alte exemple:

$$f(a,b,c) = \bar{a}b\bar{c} + ab\bar{c} + \bar{a}c + bc$$

a	b	c	f
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

- Cum facem?
  - Scriem toate combinațiile de valori ordonate
    - Atenție la simetrie!
  - Evaluăm fiecare combinație în parte

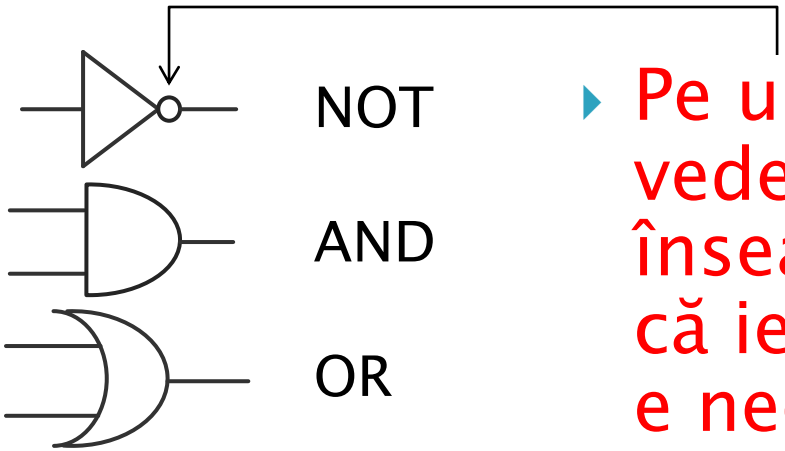
Dispozitive

- Există diverse dispozitive (elemente de comutație) ce generează rezultatele funcțiilor f_1 – f_3

$f_1(a) = \bar{a}$

$f_2(a) = a \wedge b = a \bullet b$ (AND)

$f_3(a) = a \vee b = a + b$ (OR)



- Pe unde vedeți ∘, înseamnă că ieșirea e negată!

a	ā
0	1
1	0

a	b	$a \wedge b$	$a \vee b$
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	1

Teoremele algebrei booleene (4)

- Există operatori fundamentali și operatori derivați (care se exprimă cu ajutorul celor fundamentali):
- Se definește operatorul relațional „-->” a.î. pentru fiecare pereche de elemente a, b ∈ B, a --> b este echivalent cu a • $\bar{b} = 0$.
- T9: Relația „-->” este reflexivă: a --> a, ∀ a ∈ B
- T10: Relația „-->” este tranzitivă: dacă a --> b și b --> c, atunci a --> c
- T11: Dacă x --> y şi x --> z, atunci x --> y • z
- T12: Dacă x --> y, atunci x --> y + z, ∀ z ∈ B
- T13: Dacă $\bar{x}$ --> $\bar{y}$, atunci y --> x
- T14: Dacă x --> y şi y --> x, atunci x = y

Monom

- Un monom de n variabile x_1, x_2, ... x_n este o expresie de forma ${x_1}^{e_1} \cdot {x_2}^{e_2} \cdot \ldots \cdot {x_n}^{e_n}$, unde

$${x_i}^{e_i} = \begin{cases} x_i, & dacă \quad e_i = 1 \\ !\,x_i, & dacă \quad e_i = 0 \end{cases}$$

- Ex: având variabilele a,b,c, se pot crea $2^3 = 8$ monoame diferite: !a!b!c, !a!bc, !ab!c, !abc, a!b!c, a!bc, ab!c și abc

Forma Disjunctivă

- Reprezentarea funcției sub formă de monoame conduce la forma disjunctivă a funcției
- Ex: $f_6 = ab + a!b$ – funcție disjunctivă ce depinde de 2 variabile (a, b)
- Dar $f_6 \overset{P_3}{=} a \bullet (b + !b) \overset{P_4}{=} a \bullet 1 \overset{P_2}{=} a$
- Câte circuite sunt necesare pentru implementare:
  - a) în forma inițială? 1 NOT, 2 ȘI, 1 SAU
  - b) în forma finală? Nimic
- Iată de ce sunt utile teoremele și proprietățile!

Forma Disjunctivă Normală (FDN)

- $f_7(a, b, c) = ab + a!bc$ → uneori ne ajută ca funcțiile să fie într-o formă omogenă → FDN
- FDN cuprinde monoame ce depind de toate variabilele de care depinde funcția
- Monoamele ce depind de toate variabilele de care depinde funcția (adică sunt complete) se numesc mintermi.
- $ab = ab \bullet 1 = ab\,(c + !c) = abc + ab!c$
- $f_7(a, b, c) = abc + ab!c + a!bc$ FDN

Forma Disjunctivă Normală (FDN)

- De ce am face asta? (de ce să complicăm funcțiile)?
  - La unele circuite e posibil să fie mai ușor așa
  - Prin FDN se trece foarte ușor de la tabel de adevăr la forma algebrică și invers

a	b	c	f7 = ab+a!bc
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

- $f_7(a, b, c) = abc + ab!c + a!bc$
- În forma inițială e mai dificil să evaluăm decât în FDN, pentru că f = 1 dacă cel puțin unul din mintermi e 1 → x + 1 = 1
- Se poate și invers: se dă tabelul de adevăr și se cere forma algebrică a funcției → FDN cu termenii care au 1 în tabel

Forma Disjunctivă Normală Completă (FDNC)

- Def: Suma tuturor mintermilor ne dă FDNC.
- Ex: $f_8(a,b) = !a!b + !ab + a!b + ab = 1$
- T15: Orice funcție reprezentată ca FDNC are valoarea 1.
  - Dem: recursiv, pentru modul general SAU particular, prin factor comun SAU prin tabel de adevăr – peste tot e 1, deci valoarea e 1
- Două funcții se consideră echivalente dacă iau aceleași valori pentru fiecare combinație de valori ale variabilelor de intrare
- T16: Fiecare funcție dintr-o algebră booleană este echivalentă cu o FDN (oricât de complicată e o expresie, ea poate fi adusă la FDN)

Forma Conjunctivă Normală (FCN)

- Suma logică a tuturor variabilelor în forma lor directă sau negată se numește maxterm.
- Ex: a + b + !c
- O funcție este reprezentată în FCN dacă este reprezentată ca produs logic al maxtermilor săi.
- Ex: $f_9(a, b, c) = (a + b)(a + c)$ nu e FCN
  $f_{10}(a, b) = (a + b)(a + !b)$ e FCN
- FDN: sumă de mintermi
- FCN: produs de maxtermi

FDN ⇔ FCN

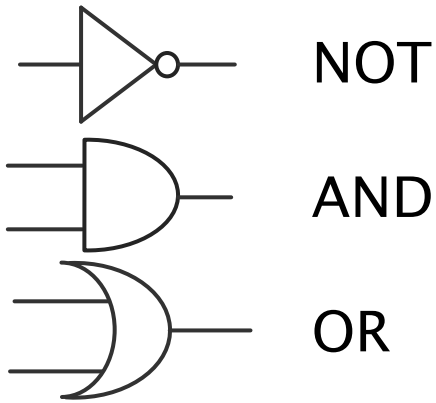
- Pentru a trece dintr-o formă în alta se folosesc regulile lui de Morgan
- Ex:

$$f_{10}(a,b) = (a+b)\left(a+\bar{b}\right) = \overline{\overline{(a+b)(a+\bar{b})}} = \overline{\bar{a}\bar{b} + \bar{a}b}$$

- Se mai poate apela și la desfacerea parantezelor:
- Ex: $f_{10}(a, b) = a + ab + a!b + b!b = a + a \bullet (b+!b) = a + a \bullet 1 = a$

Set complet de operatori

- Introducerea de operatori nu se face oricum, ci trebuie să respecte niște reguli.
- Un set de operatori se consideră complet din punct de vedere funcțional, dacă orice funcție booleană poate fi reprezentată prin intermediul acestor operatori.
- Ex: (!, •, +) = set complet fundamental de operatori.

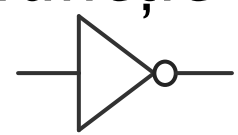


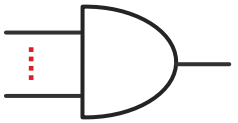
Seturi complete de operatori

- (!, +) = set complet de operatori: de Morgan

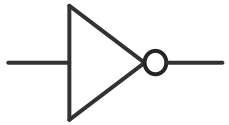
  a • b = !(!a + !b)
- (!, •) = set complet de operatori: de Morgan

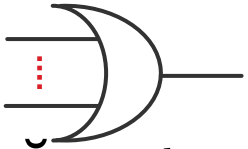
  a + b = !(!a • !b)
- ∀ funcție poate fi implementată folosind exclusiv


NOT

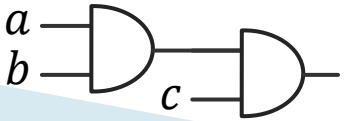

AND

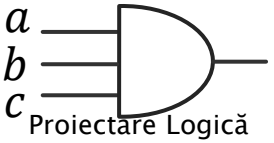
SAU

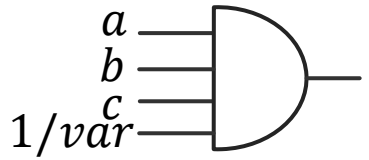

NOT


OR

- Observație: am pus ... pentru că pot avea și mai multe intrări, nu fix 2. În practică pot avea 2, 3, 4 sau 8 intrări.
- Ex: a • b • c cu ȘI cu 2, 3, 4 intrări: abc = (ab)c







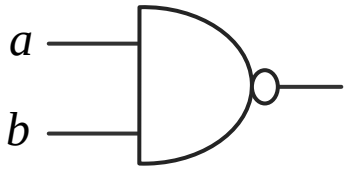
Seturi complete de operatori

- Operator nou: NAND – „ȘI-NU” (↑): a ↑ b = !(a • b)
- Operator nou: NOR – „SAU-NU” (↓): a ↓ b = !(a + b)
- a • b = !(a ↑ b) → • se poate exprima prin (!, ↑)
- Dar (!, •) era set complet de operatori → (!, ↑) e set complet de operatori
- Similar, (!, ↓) e set complet de operatori pentru că (!, +) e set complet de operatori, iar a + b = !(a ↓ b)
- Fie a = b: a ↑ a = !(a • a) = !a → ! se poate exprima prin ↑ și cum (!, ↑) e set complet de operatori → (↑) e set complet de operatori
- Similar, a ↓ a = !(a + a) = !a → (↓) e set complet de operatori

Seturi complete de operatori

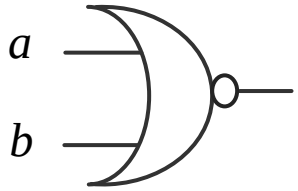
- Datorită faptului că NOR și NAND reprezintă seturi complete de operatori, există elemente de comutație pentru ele:

a	b	$a \uparrow b$
0	0	1
0	1	1
1	0	1
1	1	0



$f = \overline{a \cdot b}$ NAND

a	b	$a \downarrow b$
0	0	1
0	1	0
1	0	0
1	1	0



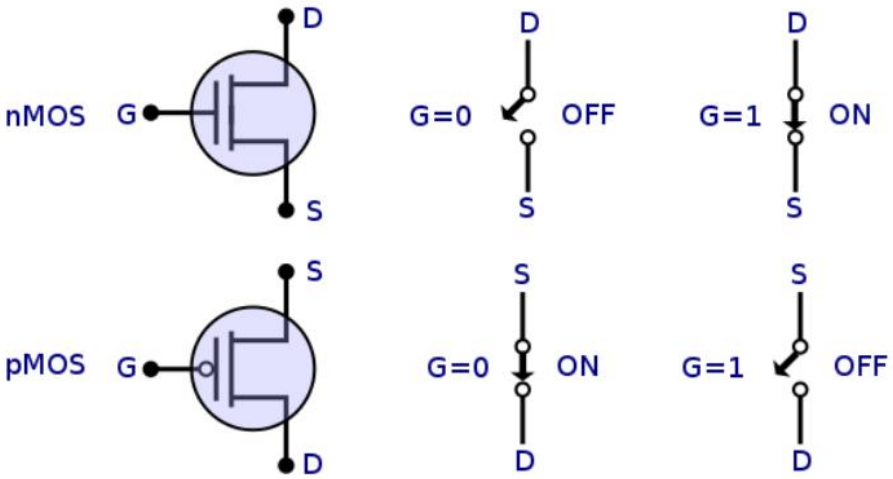
$f = \overline{a + b}$ NOR

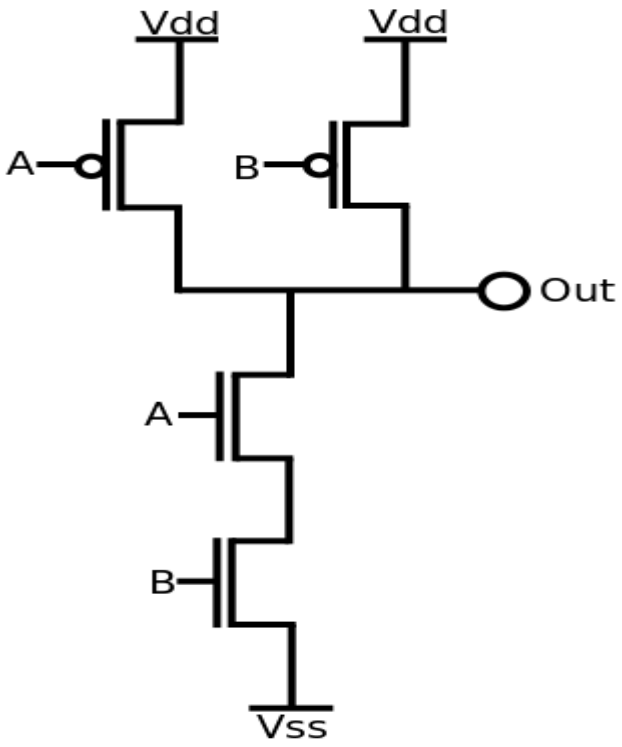
Seturi complete de operatori

- Investigăm „-->” : a --> b => a • !b = 0 →
- a --> b = !(a • !b) → !(a --> b) = a • !b
- Notăm !b = B → a • B = !(a --> !B) → • se poate exprima prin ! și -->
- Cum (!, •) = set complet de operatori
  → (!, -->) = set complet de operatori
- Concluzie:
  - (!, •), (↑), (!, +), (↓), (!, -->) = seturi complete de operatori
  - (!, •, +), (!, ↑), (!, ↓) = seturi supracomplete de operatori

Implementarea circuitului NAND (1)

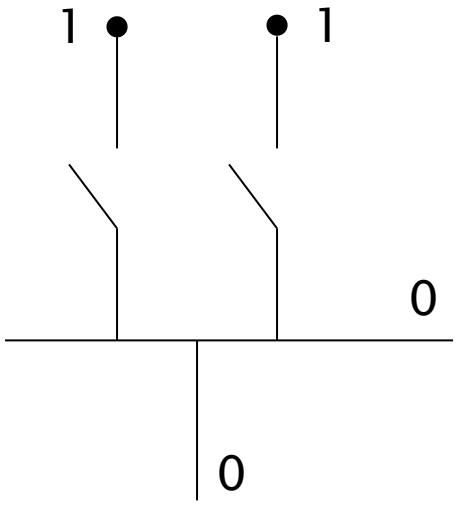
- Vă mai amintiți tranzistorii?
- Fie următoarea combinație de tranzistori în care A = 1, B = 1, Vdd = 1, Vss = 0 (amintiți-vă, sunt doar voltaje!!)





Implementarea circuitului NAND (2)

- Conform echivalenței tranzistor – întrerupător discutată în cursul anterior (tranzistorul = întrerupător acționat prin tensiunea aplicată pe G), schema anterioară va genera Out = 0 pentru A = 1 și B = 1.
- Datorită legii lui Ohm – între Out și Vss nu "dispare" niciun fel de voltaj, pentru că nu am o rezistență pe care să se disipe, iar circuitul este deconectat de Vdd, deci aceasta nu influențează



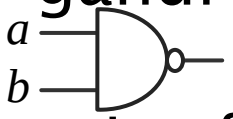
Implementarea circuitului NAND (3)

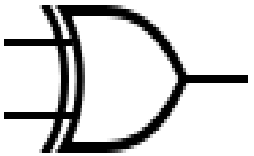
- Dacă continuăm să analizăm circuitul dând toate valorile posibile pentru A și B (0/0, 0/1, 1/0, 1/1) obținem tabelul de la NAND
- Practic, am construit o funcție logică din tranzistori
- În plus, cum NAND reprezintă un set complet de operatori → combinând mai multe circuite cu tranzistori de acest fel (NAND) se poate obține orice expresie logică se dorește

A	B	out
0	0	1
0	1	1
1	0	1
1	1	0

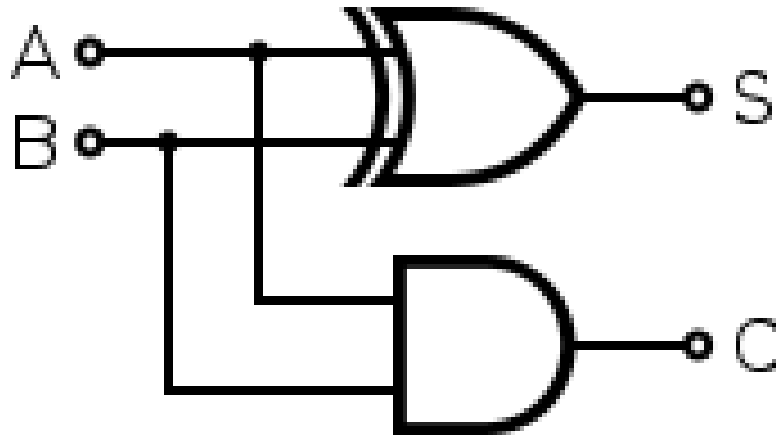
Mai multe informații despre functii logice ca circuite puteți găsi aici: https://en.wikipedia.org/wiki/NAND_gate

Implementarea circuitului NAND (4)

- Pentru a nu desena atât de mult și pentru a nu ne gândi la tranzistori, circuitul anterior îl notăm cu 

și nu ne batem capul cu tranzistori sau orice fel de fenomen fizic :)
- Notă: Tehnica aceasta de abstractizare (înlocuire cu o cutie neagră, un simbol) este extrem de utilizată în inginerie (funcțiile în C sunt un alt exemplu, în software).
- Cum am construit poarta NAND putem construi și alte porți, de exemplu AND sau XOR (are valoarea 1 doar când intrările sunt diferite). 

De ce facem toate astea?



Inputs		Outputs		
A	B	C	S	
0	0			0
0	1			1
1	0			1
1	1			2

- Combinând ceea ce am învățat, am reușit să construim un circuit care realizează operația de adunare în binar pe 1 bit.

Concluzie

- Am pornit cu niște lucruri foarte simple (niște întrerupătoare), din care am făcut lucruri ceva mai complicate (porți logice), din care am făcut lucruri și mai complicate (un circuit care adună numere pe 1 bit).
- Similar putem să facem un circuit care să adune numere pe 64 de biți (cam cum fac procesoarele actuale :D).
- La fel putem face circuite care să selecteze o intrare sau alta (ca if-urile din C), sau să facă operații logice (avem deja porțile logice), sau să memoreze biți.
- În final, putem construi întreaga UAL folosind combinații de astfel de circuite de adunare, selecție, memorare, etc.

ÎNTREBĂRI?